

PROGETTO INFORMATICA CON LABORATORIO

Equazione del calore 2D e riordinamento nelle fattorizzazioni Cholesky

Roberto Grossi, Leonardo Robol

Anno Accademico 2025/2026

1 Equazione del calore

Siamo interessati a risolvere il problema differenziale

$$\begin{cases} \kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0 & (x, y) \in [0, 1]^2 \\ u(x, y) \equiv u_0(x, y) & (x, y) \in \partial[0, 1]^2 \end{cases}$$

che determina la temperatura di equilibrio nel punto (x, y) di una piastra quadrata 2D in cui la temperatura è nota sul bordo (è la funzione $u_0(x, y)$), ed è presente una sorgente di calore interna con densità di potenza ceduta per unità di area $f(x, y) \geq 0$. Consideriamo $\kappa = 0.01$ e $f(x, y) = e^{-10(x^2+y^2)}$, e $u_0(x, y) \equiv 0$.

1.1 Discretizzazione e riordinamento del dominio

Consideriamo la griglia di punti (x_i, y_j) per $i, j = 0, \dots, N + 1$ della forma

$$x_i = i \cdot h, \quad y_j = j \cdot h, \quad h = \frac{1}{N + 1}.$$

Possiamo definire il *grafo di adiacenza* di questa discretizzazione del dominio $[0, 1]^2$ come il grafo $G = (V, E)$ in cui i nodi $V = \{(x_i, y_j)\}$ sono tutti i punti (x_i, y_j) *interni* del dominio (quelli che non stanno sul bordo), ed esiste un arco

$$(x_i, y_j) \longleftrightarrow (x_k, y_l) \iff i = k \text{ e } |j - l| = 1, \text{ oppure } j = l \text{ e } |i - k| = 1$$

In pratica, due nodi sono collegati solo se sono vicini nella griglia. La struttura è evidenziata nella Figura 1. Possiamo notare che, essendo questa relazione simmetrica, il grafo risultante è non diretto.

Task 1. Scrivere un codice C++ che prende in input N e genera i seguenti file di testo:

- **coords.txt** Un file che contiene la lista di tutti i punti interni del dominio con il loro indice i, j e le loro coordinate spaziali x, y ; ogni riga deve avere la forma

```
n    i    j    x    y
```

dove n è un indice progressivo.

- **connectivity.txt** Un file che contiene la lista di archi del grafo, con righe della forma

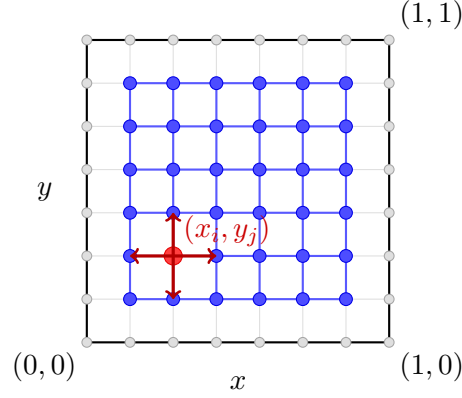


Figura 1: Punti esterni ed interni del dominio, segnati rispettivamente in grigio ed in blu, e punti adiacenti, evidenziati in rosso per un punto di esempio (x_i, y_j) .

e n1 n2

dove $(x_i, y_j) \longleftrightarrow (x_k, y_l)$ sono punti adiacenti che corrispondono all'elemento di indice n_1 ed n_2 dentro il file `coords.txt`, ed e è un indice progressivo a partire da 0 utilizzato per numerare gli archi.

Task 2. Si scriva un codice C++ che, presi in input i file generati nel punto precedente, determini una partizione dei nodi $V = V_1 \cup V_2 \cup V_S$ tale che:

- $V_1 \cap V_2 = \emptyset$;
- V_S è di cardinalità bassa;
- V_1 e V_2 hanno approssimativamente la stessa cardinalità.

Determinare un separatore bilanciato ottimo del grafo non è, in generale, economico. Una buona euristica geometrica consiste nel separare il dominio spaziale in due parti più o meno uguali. Ad esempio, è possibile selezionare $\hat{x}$ che coincida con uno degli x_i e prendere gli insiemi:

$$\begin{aligned} V_1 &= \{(x_i, y_j) \mid x_i < \hat{x}\}, \\ V_2 &= \{(x_i, y_j) \mid x_i > \hat{x}\}, \\ V_S &= \{(x_i, y_j) \mid x_i = \hat{x}\} \end{aligned}$$

La stessa cosa si può fare selezionando $\hat{y}$. Applicando questa idea ricorsivamente si ottiene un ordinamento di tipo *nested dissection*. Il codice deve:

- Riordinare i nodi del grafo listando i nodi in V_1 , seguiti da quelli in V_2 , ed infine da quelli in V_S ;
- Applicare la stessa idea ricorsivamente a V_1 ed V_2 , dividendo una volta con $\hat{x}$ ed una con $\hat{y}$.
- Salvare l'ordinamento finale dei nodi su un file `ordering.txt` con la struttura

m n

dove m sono interi da 0 a $N^2 - 1$ e rappresentano i nuovi indici dei nodi dopo il riordinamento, mentre n è l'identificatore originale del nodo nel file `coords.txt`.

1.2 Soluzione dell'equazione

Consideriamo l'approssimazione alle differenze finite

$$\kappa \frac{\partial^2 u}{\partial x^2}(x_i, y_j) + \kappa \frac{\partial^2 u}{\partial y^2}(x_i, y_j) \approx \frac{\kappa}{h^2} \left[u(x_{i-1}, y_j) + u(x_{i+1}, y_j) + u(x_i, y_{j-1}) + u(x_i, y_{j+1}) - 4u(x_i, y_j) \right]$$

Indichiamo con $u_{i,j}$ l'approssimazione della soluzione nel punto (x_i, y_j) , e si consideri la matrice A del sistema lineare ottenuto imponendo, per tutti punti interni del dominio $u_{i,j}$, che valga

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0$$

Task 3. Costruire un codice C++ che produce i file `A.txt` e `rhs.txt` che contengano tutte le entrate della matrice A e quelle del vettore dei termini noti del sistema lineare sopra. Il codice deve poter considerare le incognite del sistema lineare ordinate come specificato dal file `ordering.txt`, oppure con l'ordinamento "naturale" del file `coords.txt`. Ogni riga di `A.txt` deve contenere gli indici

i j A[i,j]

rispetto al nuovo ordinamento. Ogni riga di `rhs.txt` deve contenere solo il valore del termine noto che corrisponde alla stessa riga nel file per A . I valori di u sui punti di bordo non sono incognite del sistema lineare: i loro eventuali contributi devono essere incorporati nel termine noto.

Opzionale: aggiungere al codice la possibilità di scegliere una funzione $u_0(x, y) \neq 0$; questo richiederà di cambiare il right hand side del sistema lineare in modo coerente.

Task 4. Scrivere un notebook Python che:

- Legge i file `A.txt` e `rhs.txt` e salva la matrice *sparsa* in formato CSC. La matrice può essere letta in qualunque formato sparso e poi convertita in CSC usando le funzioni di SciPy.
- Calcola la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore (la matrice A è definita negativa, per cui dobbiamo fattorizzare $-A$); è possibile ottenere questa fattorizzazione con il seguente codice Python, che si appoggia a sua volta alla libreria CHOLMOD. Questa funzione richiede di installare il pacchetto `scikit-sparse`.

```
import numpy as np
from scipy.sparse import csc_matrix, tril
from sksparse.cholmod import cholesky

def my_cholesky(A):
    factor = cholesky(A, order="natural")
    return csc_matrix(factor[0])
```

- Risolve il sistema lineare che ci interessa usando la funzione per risolvere sistemi triangolari `scipy.sparse.linalg.spsolve_triangular`.

Task 5. Risolvere il sistema lineare per i valori di $N = 32, 64, \dots, 1024$. Confrontare i tempi e il numero di entrate non-zero richieste con l'ordinamento naturale e con quello ottenuto con il riordinamento proposto. Nel notebook, fornire dei plot della soluzione, e delle immagini della struttura della matrice usando la funzione `matplotlib.pyplot.spy`.

1.3 Nota importante sulla consegna del progetto

La consegna del solo codice in C++/Python non è sufficiente per superare l'esame. È necessario documentare il **processo** che ha portato alla realizzazione del codice, utilizzando un repository su `github.com` nel quale siano effettuati commit e push periodici, in modo da conservare gli snapshot della progressione del progetto.

Prima di iniziare la scrittura del codice, occorre inserire nel repository un documento contenente una descrizione sufficientemente dettagliata di come si intende procedere nello sviluppo. Il documento partirà da uno schema iniziale essenziale e sarà successivamente raffinato. Anche per il documento dovranno essere effettuati vari commit e push, così da mostrare il processo concettuale attraverso cui è evoluto.

Dopo aver predisposto il documento iniziale, si potrà iniziare a scrivere il codice, continuando a effettuare commit e push periodici. È possibile sviluppare in parallelo sia il documento sia il codice, purché il documento rifletta lo stato del codice in quel momento.

La discussione del progetto richiede di fornire ai docenti l'accesso al repository `github.com`, aggiungendoli come utenti `robo1` e `robertogrossi`. Durante la discussione i docenti esamineranno i vari commit presenti nel repository e potranno chiedere chiarimenti e motivazioni sulle scelte progettuali e implementative adottate durante lo sviluppo.

È consentito utilizzare strumenti di AI generativa durante la realizzazione del progetto. Tuttavia, è necessario conoscere in modo **approfondito** il contenuto del codice prodotto e saperne **spiegare/modificare** il funzionamento. In caso contrario, il progetto non sarà considerato sufficiente per il superamento dell'esame.

Non sarà considerata valida ai fini della consegna una cronologia di commit nella quale documento e codice compaiano già nella loro forma finale o quasi, poiché ciò non permette di ricostruire il percorso progettuale e implementativo che ha portato alla realizzazione del progetto.

APPENDICE

A Struttura del documento di progetto

Il documento che accompagna il codice sorgente deve descrivere in modo chiaro e progressivo il processo di progettazione e sviluppo del software. Il documento non deve essere scritto soltanto alla fine del progetto, ma deve evolvere gradualmente insieme al codice, tramite commit e push periodici sul repository `github.com`.

La struttura del documento deve seguire un processo di raffinamento successivo. In una prima fase è sufficiente una descrizione generale dell'architettura del software e degli obiettivi principali. Successivamente il documento dovrà essere raffinato progressivamente, introducendo livelli crescenti di dettaglio fino ad arrivare a una corrispondenza precisa tra documento e codice implementato.

Il documento dovrebbe inizialmente identificare:

- i principali moduli software;
- i compiti assegnati a ciascun modulo;
- le strutture dati fondamentali;
- le principali interazioni tra moduli.

In una fase successiva, ciascun modulo deve essere descritto più dettagliatamente, specificando:

- input e output;
- specifiche funzionali;
- strutture dati utilizzate;
- complessità attesa degli algoritmi;
- eventuali invarianti mantenuti dalle strutture dati;
- dipendenze da altri moduli.

Ogni modulo può essere ulteriormente suddiviso in sotto-moduli. Anche i sotto-moduli devono essere descritti attraverso raffinamenti successivi, fino a raggiungere una descrizione sufficientemente dettagliata da corrispondere direttamente alle componenti implementative presenti nel codice sorgente.

Esempio

Un possibile raffinamento potrebbe partire da un modulo generale per la gestione della discretizzazione del dominio e del sistema lineare:

HeatEquationSolver

Tale modulo potrebbe successivamente essere raffinato nei seguenti sotto-moduli:

- generazione della griglia del dominio;
- costruzione del grafo di adiacenza;
- costruzione dell'ordinamento dei nodi;
- generazione della matrice sparsa;
- generazione del termine noto;
- esportazione dei dati su file;
- risoluzione numerica del sistema lineare;
- analisi sperimentale.

Per il sotto-modulo

costruzione del grafo di adiacenza

una possibile specifica potrebbe essere la seguente.

- **Input.** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output.** Grafo non orientato

$$G = (V, E)$$

dove:

- ogni nodo rappresenta un punto interno della griglia;

- ogni arco rappresenta una relazione di adiacenza orizzontale o verticale tra punti vicini;
- ogni nodo possiede coordinate geometriche associate.

- **Funzionalità richieste.**

- Generazione dei punti interni della griglia.
- Assegnazione di identificatori numerici progressivi ai nodi.
- Costruzione delle relazioni di adiacenza.
- Esportazione dei dati nei file `coords.txt` e `connectivity.txt`.
- Supporto a successivi riordinamenti dei nodi.

- **Strutture dati suggerite.**

- Liste di adiacenza implementate tramite `vector`.
- Strutture dati ausiliarie per la conversione tra coordinate geometriche e identificatori numerici.

- **Complessità attesa.** La costruzione del grafo dovrebbe richiedere tempo lineare rispetto al numero di nodi della discretizzazione.

A loro volta, questi sotto-moduli possono essere ulteriormente raffinati in componenti più specifiche, descrivendo classi, funzioni, strutture dati ausiliarie e dettagli implementativi. Questo approccio aiuta a non perdere il filo durante lo sviluppo del codice: occorre chiarirsi il prossimo obiettivo nel documento prima di passare alla scrittura del codice.

Il documento deve quindi rappresentare non soltanto una descrizione statica del software finale, ma soprattutto il processo concettuale che ha guidato la progettazione e l'implementazione del progetto.